

Software Performance Engineering for an E-Business Bookstore

SOFT 437

Assignment 1

Group 23

Jack Fejer (20356793)
Ben Probst (20284512)
Dylan Smith (20265570)
Sascha Patel (20341248)

Introduction

Software Performance Engineering (SPE) is a systematic and quantitative methodology used to design software systems that meet responsiveness and scalability performance objectives. The goal is to change the paradigm from focusing on functionality first and performance as an afterthought to one where performance is proactively integrated into the architecture.

Performance modelling uses metrics to define the degree to which the software system can satisfy performance objectives, including the speed of individual event responses and the system's ability to maintain that speed under increasing operational demand. The process begins with an assessment of performance risk, which identifies the impact of failing to meet performance goals on the software's effectiveness.

In the case of an e-business bookstore, performance is critical due to the competitive nature of online retail and reliance on payment servers. The system must handle a distributed architecture including Web, Application, Database, and Payment servers, each introducing latencies that could compound into a negative experience for the customer. By using SPE, the business can have a quantitative basis for evaluating tradeoffs between hardware costs, software requirements, and architectural design. Having performance objectives established early ensures developers and managers are held accountable for the systems' eventual performance.

High-level Structure

The e-business bookstore's architecture consists of four primary server components and one client access point for the users.

Component	Purpose & Interaction
Client (Browser)	The access-point for customers/users that allows them to connect to send GET requests through the web server, which responds with HTML pages for requested content.
Web server	The front-end server that handles requests and responds with static pages when requested (56% of requests), as well as handles sending dynamic requests such as search queries or logins on to the Application server.
Application server	The backend application processing and logic handler, implements business logic, manages data handling and communication between the database and payment servers for processing logins, searches, and payments.
Database server	Persistent storage for application data and user information. Manages requests from the Application server to serve data to the front-end.
Payment server	Handles payment processing for customers that purchase books on the bookstore.

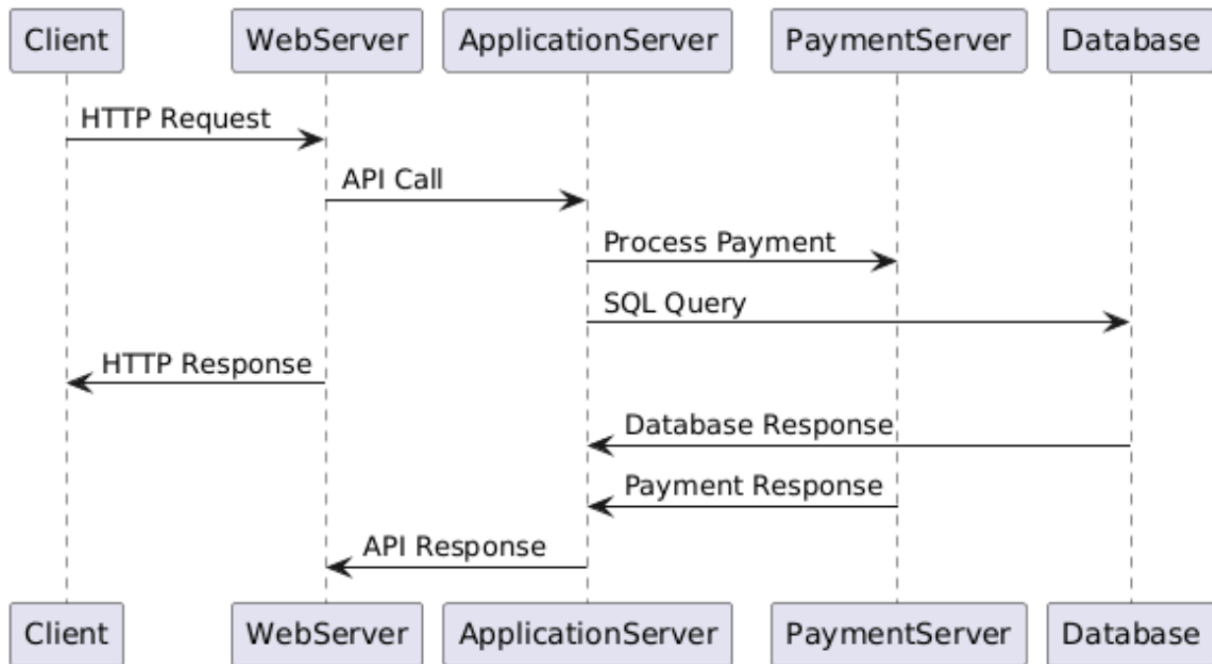


Figure 1: High-level diagram of the e-business architecture.

Detailed Design

There are three core system components: browsing, searching, and purchasing.

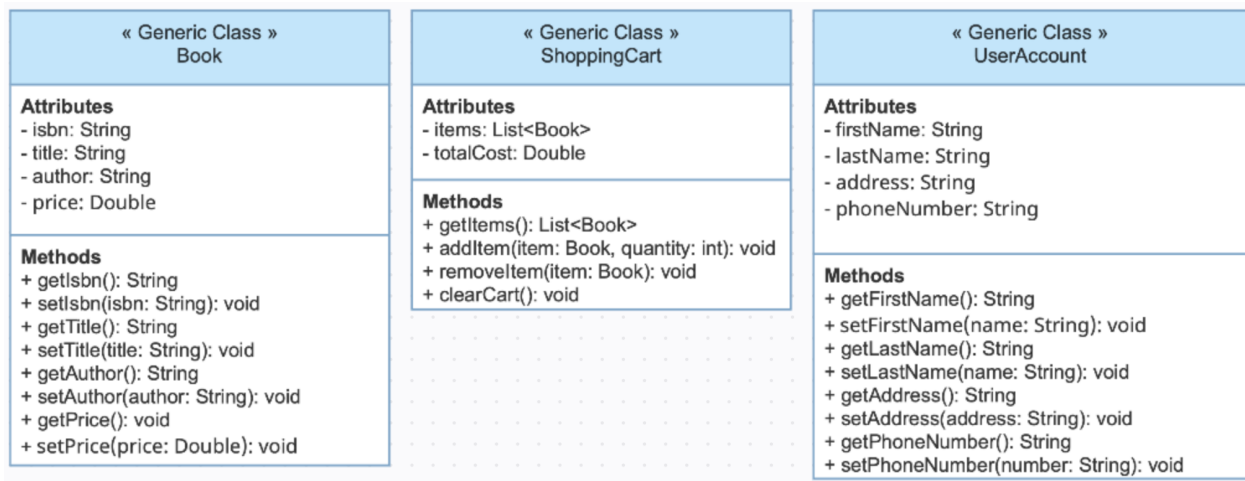


Figure 2: Class diagrams for the Book, ShoppingCart, and UserAccount classes.

These classes live inside both the Web server and Application server codebases to handle equivalent data across both the website and backend.

Critical Use Cases for Performance

Function	Visit Ratio (req/sec)	Criticality Level	Justification
Entry	1.000	Low	Standard initialization; low complexity.
Home	1.862	Medium	High frequency; impacts first impressions.
Browse	2.303	High	Highest frequency in the system; main workload.
Search	2.193	High	Second highest frequency; resource intensive.
Login	0.274	Medium	Required for transactions; includes security overhead.
Pay	0.058	High	Extreme business criticality.
Register	0.196	Low	Infrequent compared to other core functions.
Add to Cart	0.193	Medium	Necessary for conversion but lower frequency.
Select	0.919	Medium	Moderate frequency; involves detailed data retrieval.
Exit	1.000	Low	Simple session termination.

Critical Use Cases and Sequence Diagrams

In this system, there are a small number of use cases that dominate system performance. The following criteria were used to select these critical use cases: high execution frequency, significant resource consumption, and business criticality. Based on these criteria/demands, the three critical use cases being examined are: browse books, search books, checkout and payment. These use cases account for the majority of the system load and include all major processing elements.

Use Case 1: Browse Books

Occurs when the user browses for a category of books. This request is processed dynamically and requires database access.

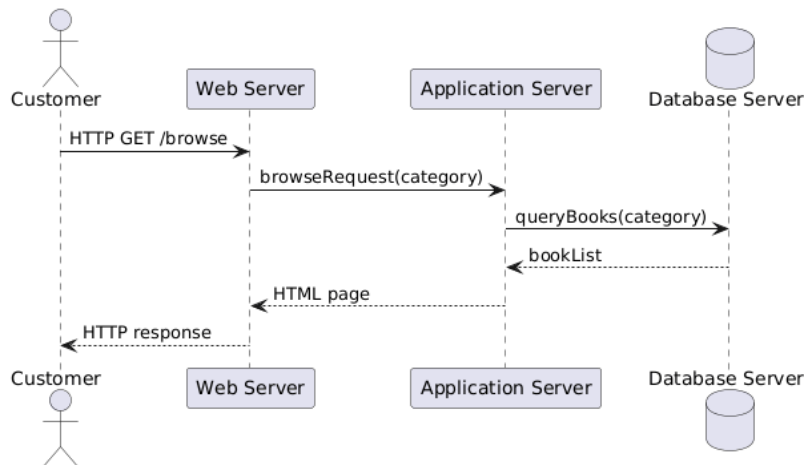


Figure 3: Use case: 'Browse Books' sequence diagram.

Use Case 2: Search Books

Occurs when the user searches for a book(s) using keywords. This request triggers the database to be searched, and the results are ranked.

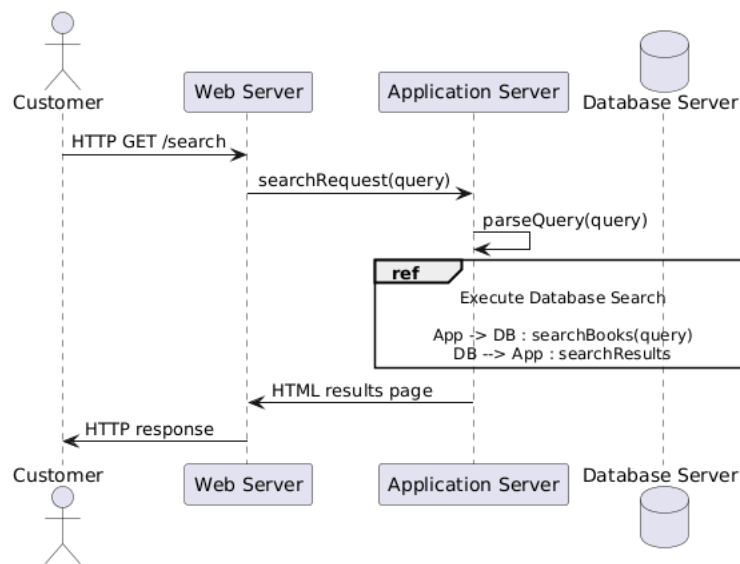


Figure 4: Use case: Search Books' sequence diagram.

Use Case 3: Checkout and Payment

Occurs when the user completes a purchase of an item. The system first retrieves cart data. It then processes the payment and updates the order.

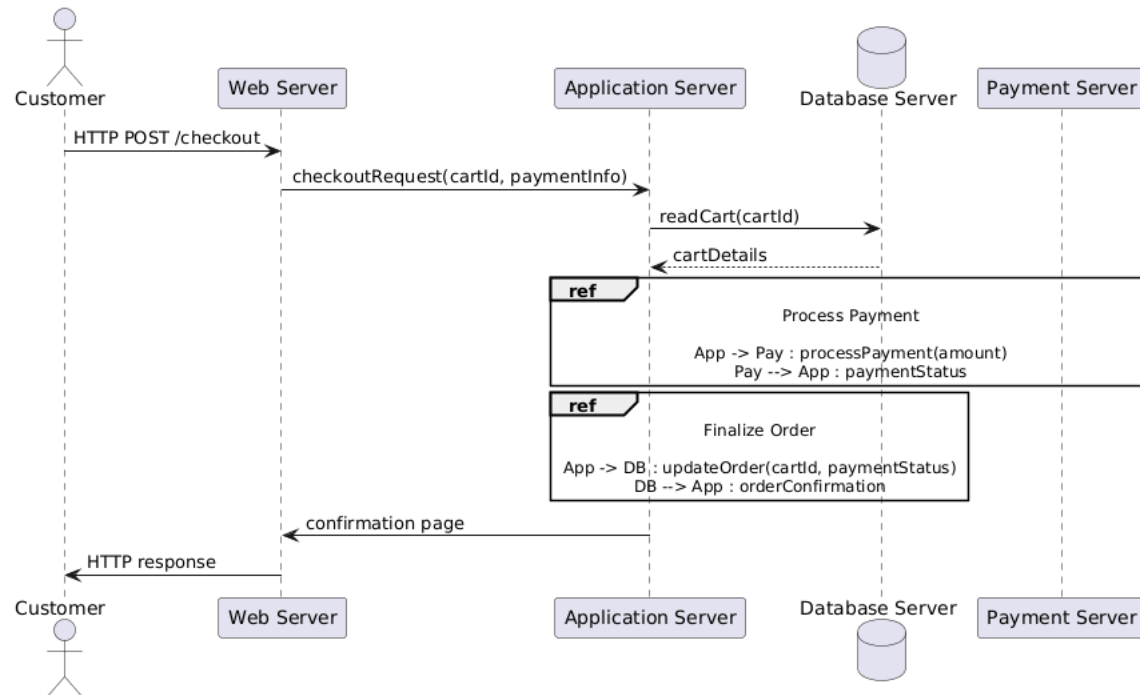


Figure 5: Use case: 'Checkout and Payment' sequence diagram.

Software Execution Models and Processing Overhead

For each critical use case, a corresponding sequence diagram exists. From this, a software execution model is formed. Sequence diagrams emphasize message ordering between components, whereas execution models describe interactions as a sequence of processing steps which use system resources.

Each node in the execution graph below represents a logical processing step executed by the software. This can include: request handling, business logic execution, database access, etc. For performance analysis, each node is accompanied by estimated demands for specific resources.

Each execution graph node is first estimated in software units (WU, DB, Msg, Delay). These estimates are then converted into device demands using the given conversion rules:

Device Service Times

Device	Service Time
CPU	0.000001 sec / KInstr
Disk	0.05 sec / I/O
Delay	0.5 sec / visit
GiNet	0.1 sec / message

Resource Conversion

Then, each device demand is multiplied by its device service time to obtain the best-case service time contribution of each node. Below is the table for resource conversion, as outlined in the assignment.

Unit	Conversion Equivalent
1 WorkUnit	25 KInstr
1 DB	500 KInstr + 4 I/O
1 Msg	25 KInstr + 1 I/O + 1 Msg
1 Delay	1 Visit

Lastly, the use-case elapsed time is the sum of the nodes' best-case times along the path.

Use Case 1: Browse Books

'Browse Books' execution graph and resource estimates per node:

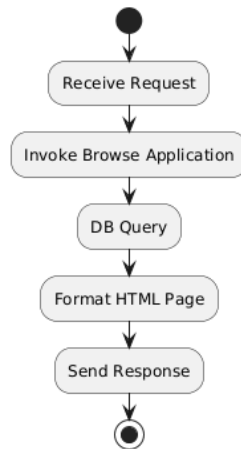


Figure 6: Use Case: 'Browse Books' execution graph.

Node	WorkUnits	DBQue	Msgs	Delay
Receive Request	1	0	1	0
Process browse Logic	2	0	0	0
Database Query	0	1	0	1
Render Page	2	0	0	0
Send Response	0	0	1	0

Totals derived by summing per-node demands of the devices:

CPU (KInstr): $50 + 50 + 500 + 50 + 25 = 675$ KInstr

Disk (I/O): $1 + 0 + 4 + 0 + 1 = 6$ I/O

GiNet (Msgs): $1 + 0 + 0 + 0 + 1 = 2$ Msg

Delay (Visits): $0 + 0 + 1 + 0 + 0 = 1$ Visit

Best-case elapsed time (device-demand × service-time):

$$T = (675 \times 10^{-6}) + (6 \times 0.05) + (2 \times 0.1) + (1 \times 0.5)$$

$$T = 1.000675 \text{ s}$$

Use Case 2: Search Books

'Search Books' execution graph and resource estimates per node:

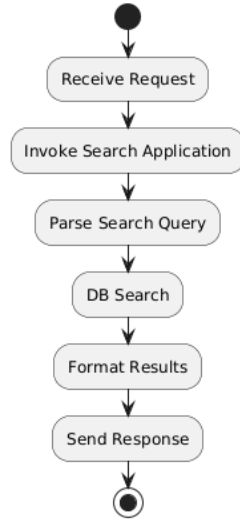


Figure 7: Use Case: 'Search Books' execution graph.

Node	WorkUnits	DB	Msgs	Delay
Receive Request	1	0	1	0
Invoke Search Application	1	0	0	0
Parse Search Query	2	0	0	0
DB Search	0	1	0	1
Format Results	3	0	0	0
Send Response	0	0	1	0

Totals derived by summing per-node demands of the devices:

CPU (KInstr): $50 + 25 + 50 + 500 + 75 + 25 = 725$ KInstr

Disk (I/O): $1 + 0 + 0 + 4 + 0 + 1 = 6$ I/O

GiNet (Msgs): $1 + 0 + 0 + 0 + 0 + 1 = 2$ Msg

Delay (Visits): $0 + 0 + 0 + 1 + 0 + 0 = 1$ Visit

Best-case elapsed time (device-demand $\times$ service-time):

$$T = (725 \times 10^{-6}) + (6 \times 0.05) + (2 \times 0.1) + (1 \times 0.5)$$

$$T = 1.000725 \text{ s}$$

Use Case 3: Checkout and Payment

'Checkout and Payment' execution graph and resource estimates per node:

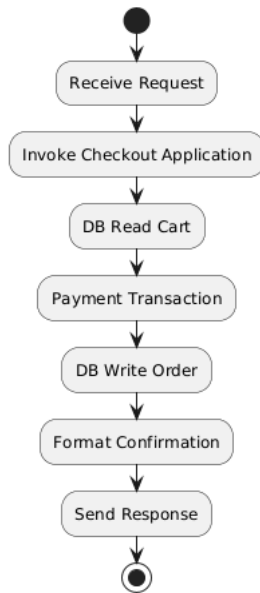


Figure 8: Use Case: 'Checkout and Payment' execution graph.

Node	WorkUnits	DB	Msgs	Delay
Receive Request	1	0	1	0
Invoke Checkout Application	1	0	0	0
DB Read Cart	0	1	0	1
Payment Transaction	1	0	1	2
DB Write Order	0	1	0	1
Format Confirmation	2	0	0	0
Send Response	0	0	1	0

Totals derived by summing per-node demands of the devices:

CPU (KInstr): $50 + 25 + 500 + 50 + 500 + 50 + 25 = 1200$ KInstr

Disk (I/O): $1 + 0 + 4 + 1 + 4 + 0 + 1 = 11$ I/O

GiNet (Msgs): $1 + 0 + 0 + 1 + 0 + 0 + 1 = 3$ Msg

Delay (Visits): $0 + 0 + 1 + 2 + 1 + 0 + 0 = 4$ Visits

Best-case elapsed time (device-demand × service-time):

$T = (1200 \times 10^{-6}) + (11 \times 0.05) + (3 \times 0.1) + (4 \times 0.5)$

$T = 2.851200$ s

Deployment Diagram

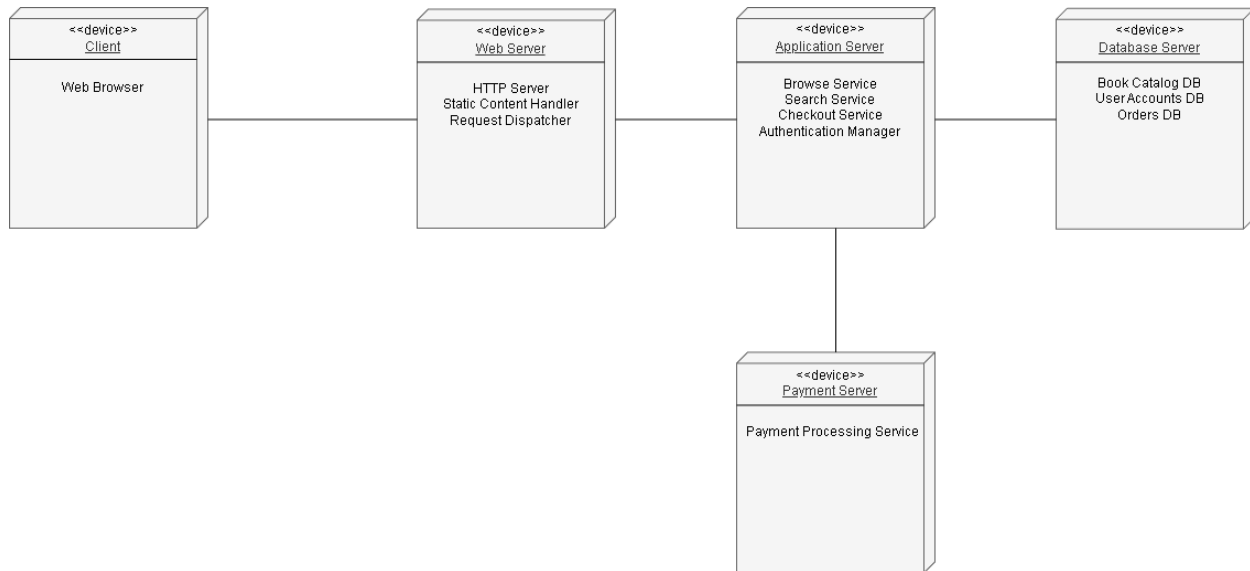


Figure 9: Deployment diagram of e-commerce bookstore

Conclusion

By using SPE techniques such as use case analysis, sequence diagrams, software execution models, and resource estimation, the online bookstore's system characteristics were analyzed to form a model and evaluate its performance prior to deployment. This approach enabled early identification of performance risks that may not be uncovered through functional design alone.

The analysis revealed "Browse Books" and "Search Books" are dominant workload drivers due to their high visit ratios. Although each request is lightweight, the frequency of these requests amplifies their impact on the web, application, and database servers. In contrast, the "Checkout and Payment" case occurs far less frequently but is still critical and introduces additional latency with database writing and communication with the external payment server.

The database server and payment server interactions were highlighted as the most performance sensitive components within the system. Database access consistently introduces CPU and I/O demands while payment processing adds a significant network delay. Under high concurrency, these factors suggest that the database could become a throughput bottleneck and the payment processing latency could negatively impact user experience.

Based on these findings, several recommendations can be made before deployment, the first being prioritization of the database with changes like indexing or query optimization to reduce disk I/O demand for high frequency browse or search operations. Also, caching of popular categories and search results would significantly reduce repeated database access. Finally, the checkout workflow should be designed to handle payment processing asynchronously, when possible, which minimizes user perceived delay.

Overall, the performance models demonstrate the value of SPE in guiding the design process early on to produce a robust, efficient application. Identifying workload drivers and bottlenecks before implementation allows for informed decisions to be made to balance cost, scalability, and responsiveness. Applying these recommendations will help ensure the bookstore meets its performance objectives while maintaining a positive user experience.